

I. Code Review & SOLID Principles

The codebase violates multiple SOLID principles, especially in the file `ItemController.cs`.

First, the class violates the Single Responsibility principle, which, besides handling HTTP requests as intended, it also handles business logic by calculating the statistics (lines 25-26) and having its own logging implementation (e.g. line 45: `Console.WriteLine($"[LOG] {DateTime.UtcNow}: GET api/item/{id} called");`).

It can be fixed by moving the statistics into a Service Layer and letting the controller handle HTTP requests and pass them to the service.

Second, it violates the Dependency Inversion principle, as the controller depends on a concrete implementation for formatting logging in methods `GetAll` and `GetById` (lines 20, 28, 45-51, 54-58). A higher-level module should not depend on low level implementation; it should depend on abstractions. A better way to handle logging would be by injecting the ASP.NET built-in `ILogger<T>` to handle logging more safely and provide better testability.

The `GetAll` method also violates the Open/Closed principle as it is not closed for modification regarding the value it returns (lines 30-39). This can be also solved by moving the statistics calculations into a separate service class.

The `Program.cs` is missing the Dependency injection registration for the interface. We should provide an implementation to `IItemReader` for it to be able to pass the `IItemReader` to the `ItemController`.

Also renamed "Item" to "Grade".